Week 3 Lab 1 — Logistic Regression Breast Cancer

5th November 2026

Description:

Breast cancer is the most common cancer amongst women in the world. It accounts for 25% of all cancer cases, and affected over 2.1 Million people in 2015 alone. It starts when cells in the breast begin to grow out of control. These cells usually form tumors that can be seen via X-ray or felt as lumps in the breast area.

The key challenge against its detection is how to classify tumors into malignant (cancerous) or benign(non-cancerous). We ask you to complete the analysis of classifying these tumors using machine learning (with logistic regression) and the Breast Cancer Wisconsin (Diagnostic) Dataset.

Acknowledgements:

This dataset has been referred from Kaggle.

Objective:

Understand the Dataset & cleanup (if required). Build a classification model to predict whether a tumour is malignant or benign, and evaluate it with appropriate metrics. Also fine-tune the hyperparameters & compare the evaluation metrics of various classification algorithms.

Step 1 Import libraries

[1]:
```
# math        -> general maths functions
# numpy       -> number handling
# pandas      -> loading and working with the data table
# matplotlib  -> plotting
# seaborn     -> the correlation heatmap
# plotly      -> the interactive charts in Step 3
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import os # Required to read environment variables
import plotly.io as pio # Requrired to export to PDF

# The variable and functions below allow the correct conversion to LaTeX tables␣
↪when exporting to PDF with nbconvert.
EXPORT_PDF = os.environ.get("EXPORT_PDF", "False").lower() == "true"
```

```
pio.renderers.default = "png" if EXPORT_PDF else "plotly_mimetype"

def correctness_html(value):
    return {
        'Yes': 'color: green; font-weight: bold;',
        'No': 'color: red; font-weight: bold;',
    }.get(value, '')


def correctness_latex(value):
    return {
        'Yes': 'color:{green}; bfseries:;',
        'No': 'color:{red}; bfseries:;',
    }.get(value, '')

def display_table(
    df,
    formats=None,
    latex_formats=None,
    html_style=None,
    latex_style=None,
    style_columns=None,
    hide_index=True,
    fit_to_page=False,
    na_rep=None,
):
    """
    Display a DataFrame as HTML and LaTeX for notebook viewing and PDF export.

    Formats values without modifying the DataFrame. The LaTeX version includes
    alternating row shading, bold headers, and escaped row and column labels.

    Parameters
    ----------
    df : pandas.DataFrame
        Table to display.
    formats : dict, optional
        Column names mapped to format strings or formatting functions.
        Used for both outputs unless overridden by latex_formats.
    latex_formats : dict, optional
        Formatting overrides for LaTeX output. Formatters that generate special
        characters must escape them, for example '%' as r'\\%'.
    html_style : callable, optional
        Function receiving a cell value and returning a CSS style string.
        Return an empty string to leave the cell unchanged.
    latex_style : callable, optional
```

```
        Function receiving a cell value and returning a pandas LaTeX style
        string, such as 'color:{green}; bfseries:;'.
    style_columns : list of str, optional
        Columns to which the styling functions apply. If None, apply them
        to all data cells. Callbacks receive original, unformatted values.
    hide_index : bool, default True
        Hide row indexes in both output formats.
    fit_to_page : bool, default False
        Shrink the LaTeX table if it exceeds the PDF text width.
        Does not affect the HTML display.
    na_rep : str or None, default None
        Text displayed for missing values. If None, use the default rendering
        without a custom replacement. Does not modify the underlying data.

    Returns
    -------
    None
        Displays HTML and LaTeX representations through IPython.display.

    Notes
    -----
    The PDF template must load xcolor and colortbl for LaTeX styling,
    and adjustbox when fit_to_page=True.
    Rerun and save the cell outputs before exporting without execution.
    """
    formats = formats or {}
    pdf_formats = {**formats, **(latex_formats or {})}

    html = df.style.format(formats, na_rep=na_rep, escape='html')
    latex = (
        df.style
        .format(pdf_formats, na_rep=na_rep, escape='latex')
        .format_index(escape='latex', axis=0)
        .format_index(escape='latex', axis=1)
    )

    if hide_index:
        html = html.hide(axis='index')
        latex = latex.hide(axis='index')

    # Apply LaTeX row shading and bold labels.
    def stripe_rows(values):
        return [
            'cellcolor:{gray!10}--rwrap;' if i % 2 == 0 else ''
            for i in range(len(values))
        ]
```

```
    def style_index(values):
        return [style + 'bfseries:;' for style in stripe_rows(values)]

    latex = (
        latex
        .apply(stripe_rows, axis=0)
        .apply_index(style_index, axis=0)
        .map_index(lambda value: 'bfseries:;', axis=1)
    )

    # Apply caller-supplied styles to selected columns.
    # If style_columns is None, apply to all data cells.
    if html_style is not None:
        html = html.map(html_style, subset=style_columns)

    if latex_style is not None:
        latex = latex.map(latex_style, subset=style_columns)

    latex_output = latex.to_latex()

    if fit_to_page:
        latex_output = (
            r"\begin{adjustbox}{max width=\linewidth}" "\n"
            + latex_output
            + "\n" r"\end{adjustbox}"
        )

    display({
        'text/html': html.to_html(),
        'text/latex': latex_output,
    }, raw=True)
```

Step 2 Import the csv file

[2]:
```
# read the breast cancer dataset and look at the first 5 rows
# 569 tumours; diagnosis is M (malignant) or B (benign)
df = pd.read_csv('data/breast-cancer.csv')
df.head()
```

[2]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	\
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	\
0	0.11840	0.27760	0.3001	0.14710	

```
1          0.08474           0.07864          0.0869             0.07017
2          0.10960           0.15990          0.1974             0.12790
3          0.14250           0.28390          0.2414             0.10520
4          0.10030           0.13280          0.1980             0.10430

   …  radius_worst  texture_worst  perimeter_worst  area_worst  \
0  …         25.38          17.33           184.60      2019.0
1  …         24.99          23.41           158.80      1956.0
2  …         23.57          25.53           152.50      1709.0
3  …         14.91          26.50            98.87       567.7
4  …         22.54          16.67           152.20      1575.0

   smoothness_worst  compactness_worst  concavity_worst  concave points_worst  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

   symmetry_worst  fractal_dimension_worst
0          0.4601                  0.11890
1          0.2750                  0.08902
2          0.3613                  0.08758
3          0.6638                  0.17300
4          0.2364                  0.07678

[5 rows x 32 columns]
```

[3]:
```
print(f"The dataset contains {df.shape[0]} rows and {df.shape[1]} columns")

df.describe()
```

```
The dataset contains 569 rows and 32 columns
```

[3]:
```
                 id  radius_mean  texture_mean  perimeter_mean    area_mean  \
count  5.690000e+02   569.000000    569.000000      569.000000   569.000000
mean   3.037183e+07    14.127292     19.289649       91.969033   654.889104
std    1.250206e+08     3.524049      4.301036       24.298981   351.914129
min    8.670000e+03     6.981000      9.710000       43.790000   143.500000
25%    8.692180e+05    11.700000     16.170000       75.170000   420.300000
50%    9.060240e+05    13.370000     18.840000       86.240000   551.100000
75%    8.813129e+06    15.780000     21.800000      104.100000   782.700000
max    9.113205e+08    28.110000     39.280000      188.500000  2501.000000

       smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
count       569.000000        569.000000      569.000000           569.000000
mean          0.096360          0.104341        0.088799             0.048919
std           0.014064          0.052813        0.079720             0.038803
```

```
min          0.052630         0.019380        0.000000             0.000000
25%          0.086370         0.064920        0.029560             0.020310
50%          0.095870         0.092630        0.061540             0.033500
75%          0.105300         0.130400        0.130700             0.074000
max          0.163400         0.345400        0.426800             0.201200

       symmetry_mean  …  radius_worst  texture_worst  perimeter_worst  \
count     569.000000  …    569.000000     569.000000       569.000000
mean        0.181162  …     16.269190      25.677223       107.261213
std         0.027414  …      4.833242       6.146258        33.602542
min         0.106000  …      7.930000      12.020000        50.410000
25%         0.161900  …     13.010000      21.080000        84.110000
50%         0.179200  …     14.970000      25.410000        97.660000
75%         0.195700  …     18.790000      29.720000       125.400000
max         0.304000  …     36.040000      49.540000       251.200000

        area_worst  smoothness_worst  compactness_worst  concavity_worst  \
count   569.000000        569.000000         569.000000       569.000000
mean    880.583128          0.132369           0.254265         0.272188
std     569.356993          0.022832           0.157336         0.208624
min     185.200000          0.071170           0.027290         0.000000
25%     515.300000          0.116600           0.147200         0.114500
50%     686.500000          0.131300           0.211900         0.226700
75%    1084.000000          0.146000           0.339100         0.382900
max    4254.000000          0.222600           1.058000         1.252000

       concave points_worst  symmetry_worst  fractal_dimension_worst
count            569.000000      569.000000               569.000000
mean               0.114606        0.290076                 0.083946
std                0.065732        0.061867                 0.018061
min                0.000000        0.156500                 0.055040
25%                0.064930        0.250400                 0.071460
50%                0.099930        0.282200                 0.080040
75%                0.161400        0.317900                 0.092080
max                0.291000        0.663800                 0.207500

[8 rows x 31 columns]
```

[4]:
```
missing = df.isna().sum()
missing = missing[missing > 0]

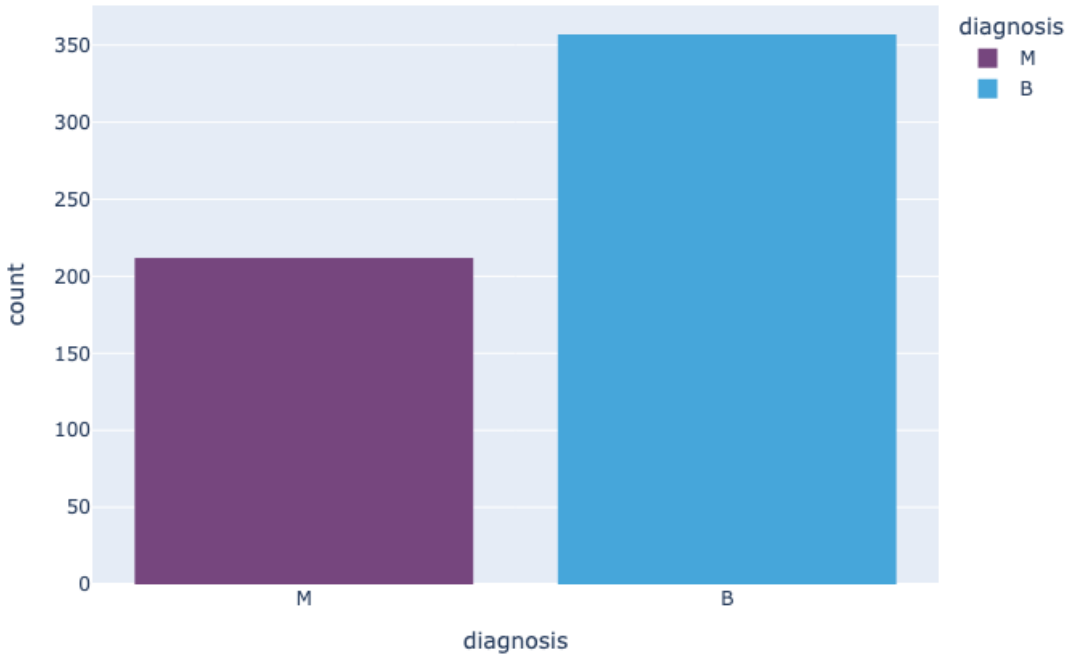
if missing.empty:
    print("No columns with missing values found.")
else:
    display(missing.to_frame('Missing values'))
```

```
No columns with missing values found.
```

Step 3.1 Visualise the dataset

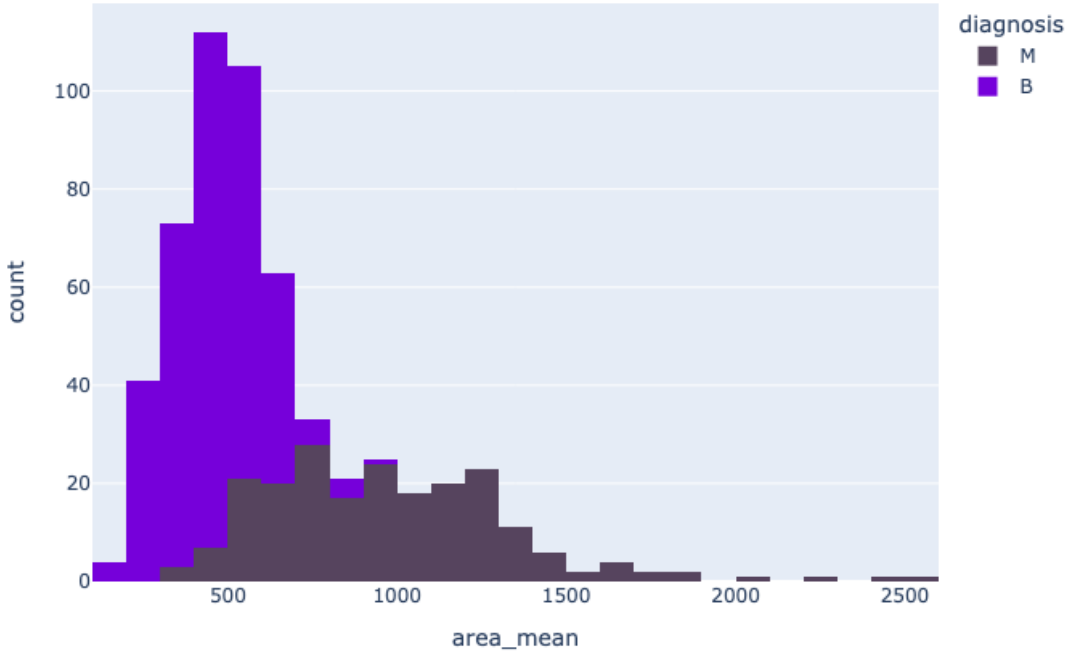
Visualise a histogram to understand the split between the diagnosis (Malignant or Benign)

[5]:
```
# interactive bar chart: how many tumours of each diagnosis?
# colour separates malignant (M) from benign (B)
px.histogram(data_frame=df, x='diagnosis',␣
  ↪color='diagnosis',color_discrete_sequence=['#76467E','#46A6DA'])
```



Step 3.2 Visualise the data set

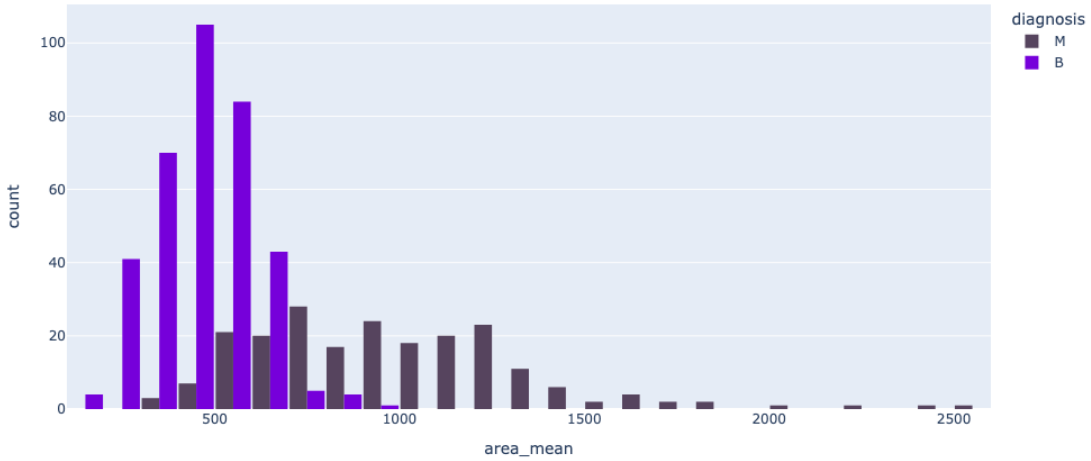
[6]:
```
# histogram of tumour area, split by diagnosis -
# malignant tumours tend to sit further right (larger areas)
px.
  ↪histogram(data_frame=df,x='area_mean',color='diagnosis',color_discrete_sequence=['#56445E',
```



[7]:
```
#This will make the output the same in both Jupyter and the PDF
print("The figure below makes the output in both Jupyter and the PDF look the␣
 ↪same.")
fig = px.histogram(
    data_frame=df,
    x='area_mean',
    color='diagnosis',
    color_discrete_map={'M': '#56445E', 'B': '#7600DA'},
    category_orders={'diagnosis': ['M', 'B']},
    barmode='group',
    width=1000,
    height=500,
)

fig.update_traces(xbins=dict(start=0, end=2600, size=100))
fig.update_layout(bargap=0.05)
fig.show()
```

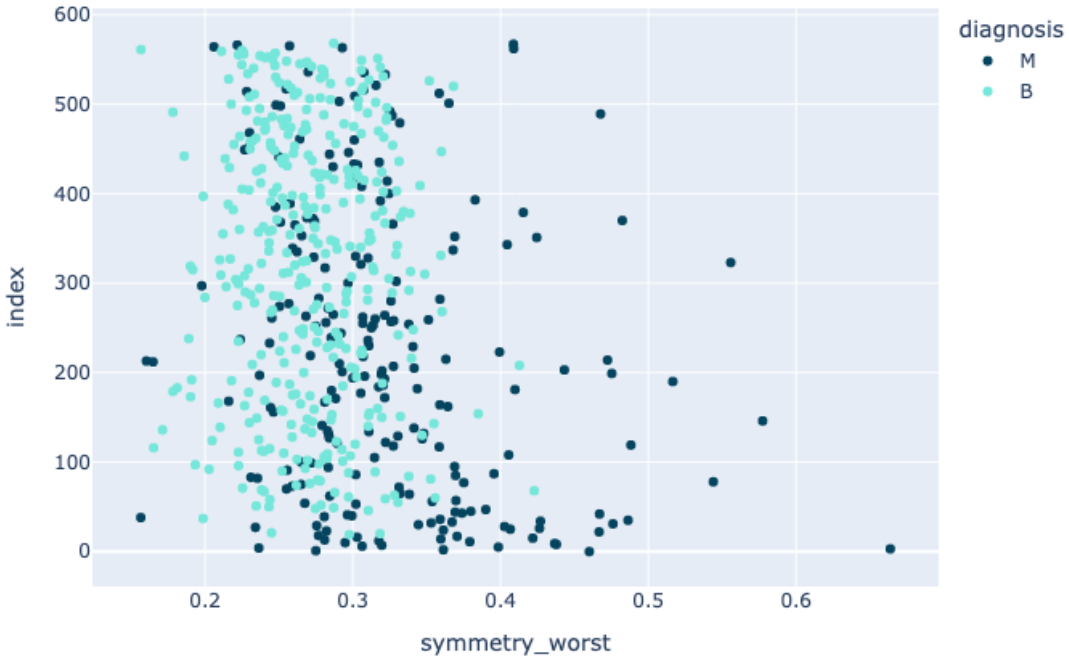
```
The figure below makes the output in both Jupyter and the PDF look the same.
```



Step 3.3 Visualise the data set

Draw a scatter plot on symmetry_worst field and understand the spread of malignancy

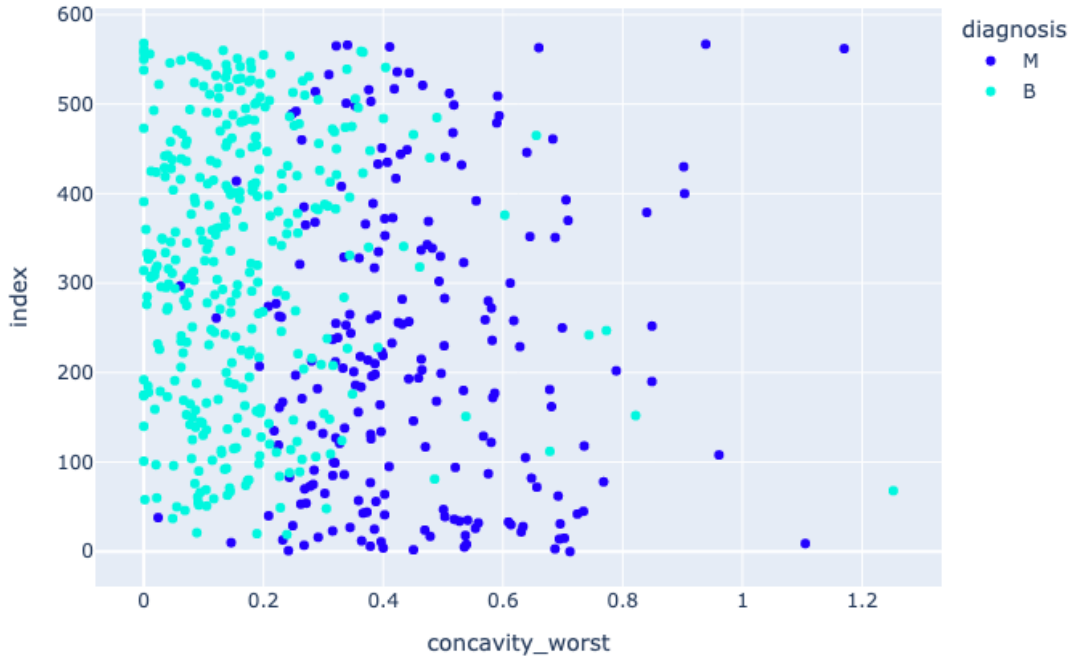
[8]:
```
# scatter of symmetry_worst for every tumour, coloured by diagnosis -
# shows how well this single feature separates the two groups
px.
  ↪scatter(data_frame=df,x='symmetry_worst',color='diagnosis',color_discrete_sequence=['#05445
```



Step 3.4 Visualise the data set

Draw a scatter plot on concavity_worst field and understand the spread of malignancy

[9]:
```
# same idea with concavity_worst - a clearer separation than symmetry
px.
  ↪scatter(data_frame=df,x='concavity_worst',color='diagnosis',color_discrete_sequence=['#2200
```



Step 4 Drop the redundant column

[10]:
```
# the id column identifies the patient but says nothing about the tumour - drop␣
 ↪it
df.drop('id', axis=1, inplace=True)
```

Step 5 Encoding the label to 0/1

The diagnosis column is 'M' or 'B', so lets convert this to 0/1. 0 is benign and 1 is malignant

[11]:
```
#print("Showing a sample of the diagnosis column before the change:")
rows_to_show = slice(16, 23)
# Even though the command below works without the copy(), it makes my intention␣
 ↪explicit: keep an independent snapshot of these values before changing them.␣
 ↪It's a reliable choice for a before-and-after comparison, even when this␣
 ↪particular operation works without it.
diagnosis_before = df.iloc[rows_to_show, 0].copy()
```

[12]:
```
# turn the diagnosis text into numbers: the comparison gives True/False,
# and astype(int) converts that to 1 (malignant) / 0 (benign)
df['diagnosis'] = (df['diagnosis'] == 'M').astype(int) #encode the label into 1/
 ↪0
```

```
# Alternative way to do it using map(). Replace the column with a newly encoded␣
 ↪Series.
#df['diagnosis'] = df['diagnosis'].map({'B': 0, 'M': 1})
```

[13]:
```
#print("Showing a sample of the diagnosis column after the change:")
diagnosis_after = df.iloc[rows_to_show, 0] # No need to copy here.

print("Showing the before and after the change")
display(pd.DataFrame({
    'diagnosis_before': diagnosis_before,
    'diagnosis_encoded': diagnosis_after,
}))
```

```
Showing the before and after the change
```

	diagnosis_before	diagnosis_encoded
16	M	1
17	M	1
18	M	1
19	B	0
20	B	0
21	B	0
22	M	1

Step 6.1 Compute Correlation

Compute pairwise correlation of columns, excluding NA/null values

[14]:
```
# correlation between every pair of columns
# (works because every column is now numeric)
corr = df.corr()
```

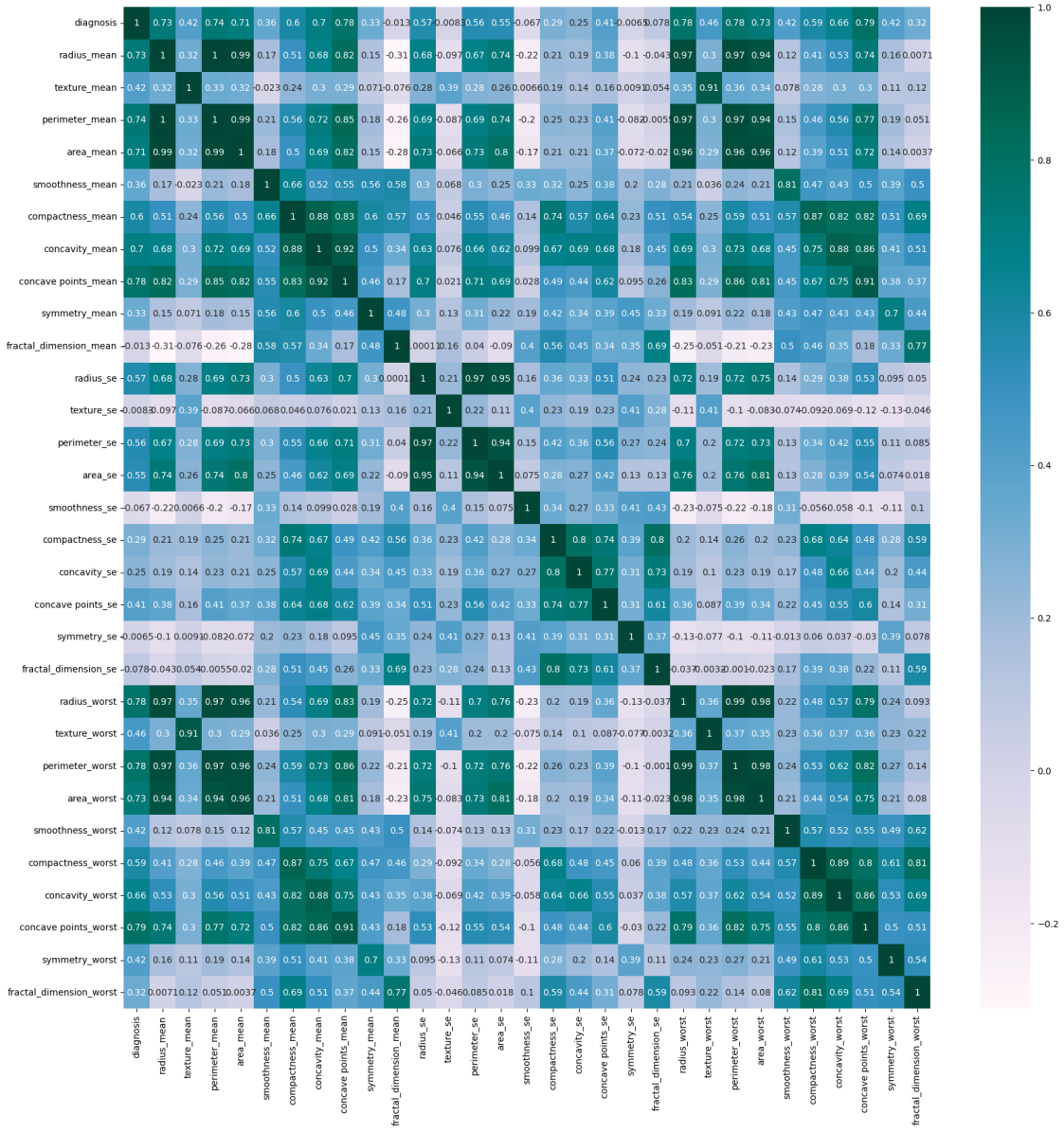
[15]:
```
# see the first few rows of the correlation matrix for the diagnosis column,␣
 ↪which is the one we're interested in on step 7
corr['diagnosis'].head()
```

[15]:
```
diagnosis          1.000000
radius_mean        0.730029
texture_mean       0.415185
perimeter_mean     0.742636
area_mean          0.708984
Name: diagnosis, dtype: float64
```

Step 6.2 Visualise the Heatmap

[16]:
```
# the full correlation matrix as a heatmap - 31 columns, so it is big;
# annot=True prints the value inside every square
plt.figure(figsize=(20,20))
sns.heatmap(corr, cmap='PuBuGn',annot=True)
plt.show()
```

```
plt.show()
```



Step 7 Select the columns

In this step get the correlated target variable and then select the correlated features with this target above 0.2. Get the names of these correlated features and get your data set ready by removing the target field.

[17]:
```
correlated_target = abs(corr["diagnosis"])                    # absolute␣
↪correlation with the target
```

```
relevant_features = correlated_target[correlated_target > 0.2]  # select␣
 ↪correlated features higher than 0.2
names = [index for index, value in relevant_features.items()]  # Get the names␣
 ↪of the features
names.remove('diagnosis')                                       # drop the␣
 ↪target itself from the results
print(names)                                                    # displaying␣
 ↪the results
```

```
['radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
'smoothness_mean', 'compactness_mean', 'concavity_mean', 'concave points_mean',
'symmetry_mean', 'radius_se', 'perimeter_se', 'area_se', 'compactness_se',
'concavity_se', 'concave points_se', 'radius_worst', 'texture_worst',
'perimeter_worst', 'area_worst', 'smoothness_worst', 'compactness_worst',
'concavity_worst', 'concave points_worst', 'symmetry_worst',
'fractal_dimension_worst']
```

Step 8 Select the columns for X and Y

[18]:
```
# X = the selected feature columns as a plain array
# y = the diagnosis (1 = malignant, 0 = benign) - what we want to predict
X = df[names].values
y = df['diagnosis'].values
```

Step 9 Split the dataset into training and testing data sets

[19]:
```
# split into training (80%) and test (20%) sets
# random_state=0 -> same split every run
# stratify=y     -> keeps the malignant/benign ratio the same in both sets
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=0, stratify=y)
```

Step 10 Normalise the data

0.0.1 Standardisation without data leakage

The scaling is performed correctly: the data is split before scaling, and the scaler learns its parameters from the training set only.

- `scaler.fit_transform(X_train)` learns each feature's training mean and standard deviation, then scales the training values.
- `scaler.transform(X_test)` reuses those stored parameters to scale the test values. It does not learn anything from the test set.

Both sets use the same transformation:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

The test features do not necessarily have mean 0 and standard deviation 1. This is expected because they are scaled using **training statistics**, not their own.

If cross-validation is added later, place the scaler and classifier inside a `Pipeline` and pass the original, unscaled training features. This ensures that scaling is learned separately within each fold.

[20]:
```
# standardise the features: rescale each column to mean 0, standard deviation 1,
# so features measured in hundreds don't drown out features measured in decimals
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# fit_transform on TRAIN: learn the mean and std, then rescale
X_train = scaler.fit_transform(X_train)

# transform only on TEST: reuse the means and stds learned from the training set
X_test = scaler.transform(X_test)
```

Step 11 Apply the Logistic Regression

In this step we apply the logistic regression from the sklearn library and fit the data of X_train and y_train

Source: LogisticRegression

```
class sklearn.linear_model.LogisticRegression(penalty='deprecated', *, C=1.0,
l1_ratio=0.0, dual=False, tol=0.0001, fit_intercept=True, intercept_scaling=1,
class_weight=None, random_state=None, solver='lbfgs', max_iter=100, verbose=0,
warm_start=False, n_jobs=None)
```

```
fit(X, y, sample_weight=None)
```

- `X: {array-like, sparse matrix} of shape (n_samples, n_features)`

  Training vector, where n_samples is the number of samples and n_features is the number of features.

- `y: array-like of shape (n_samples,)`

  Target vector relative to X.

- `sample_weight: array-like of shape (n_samples,) default=None`

  Array of weights that are assigned to individual samples. If not provided, then each sample is given unit weight.

[21]:
```
# create the Logistic Regression model and fit it to the training data
from sklearn.linear_model import LogisticRegression
classifier = LogisticRegression(random_state = 0)
classifier.fit(X_train, y_train)
```

[21]:
```
LogisticRegression(random_state=0)
```

Step 12 Confusion Matrix

In this step we evaluate the model on the test set. The confusion matrix shows the four possible outcomes — true and false positives and negatives — exactly as covered in our lesson.

0.0.2 Accuracy

Accuracy is the proportion of predictions that are correct:

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- **TP (true positives):** positive cases correctly predicted as positive.
- **TN (true negatives):** negative cases correctly predicted as negative.
- **FP (false positives):** negative cases incorrectly predicted as positive.
- **FN (false negatives):** positive cases incorrectly predicted as negative.

For example, 90 correct predictions out of 100 give an accuracy of **0.90**, or **90%**.

Accuracy counts both types of error equally. It does not show whether mistakes are mainly false positives or false negatives; the confusion matrix reveals that distinction.

[22]:
```
# from sklearn.metrics import confusion_matrix, accuracy_score
# import sklearn

# y_pred = classifier.predict(X_test)
# sklearn.metrics.accuracy_score(y_test,y_pred)
# sklearn.metrics.accuracy_score(y_test,y_pred)
# cmatrix = confusion_matrix(y_test, y_pred)
# conf_matrix = pd.DataFrame(data=cmatrix, columns=['Predicted:0','Predicted:
↪1'],index=['Actual:0','Actual:1'])
# plt.figure(figsize = (10,8))
# sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='YlOrRd')
# confusion_mat = confusion_matrix(y_test, y_pred)
# print(confusion_mat)
# accuracy_score(y_test, y_pred)

# Import the two evaluation tools we need from scikit-learn
from sklearn.metrics import confusion_matrix, accuracy_score

# Ask the trained model to predict a diagnosis (0 or 1) for every
# tumour in the test set - data it has never seen during training
y_pred = classifier.predict(X_test)

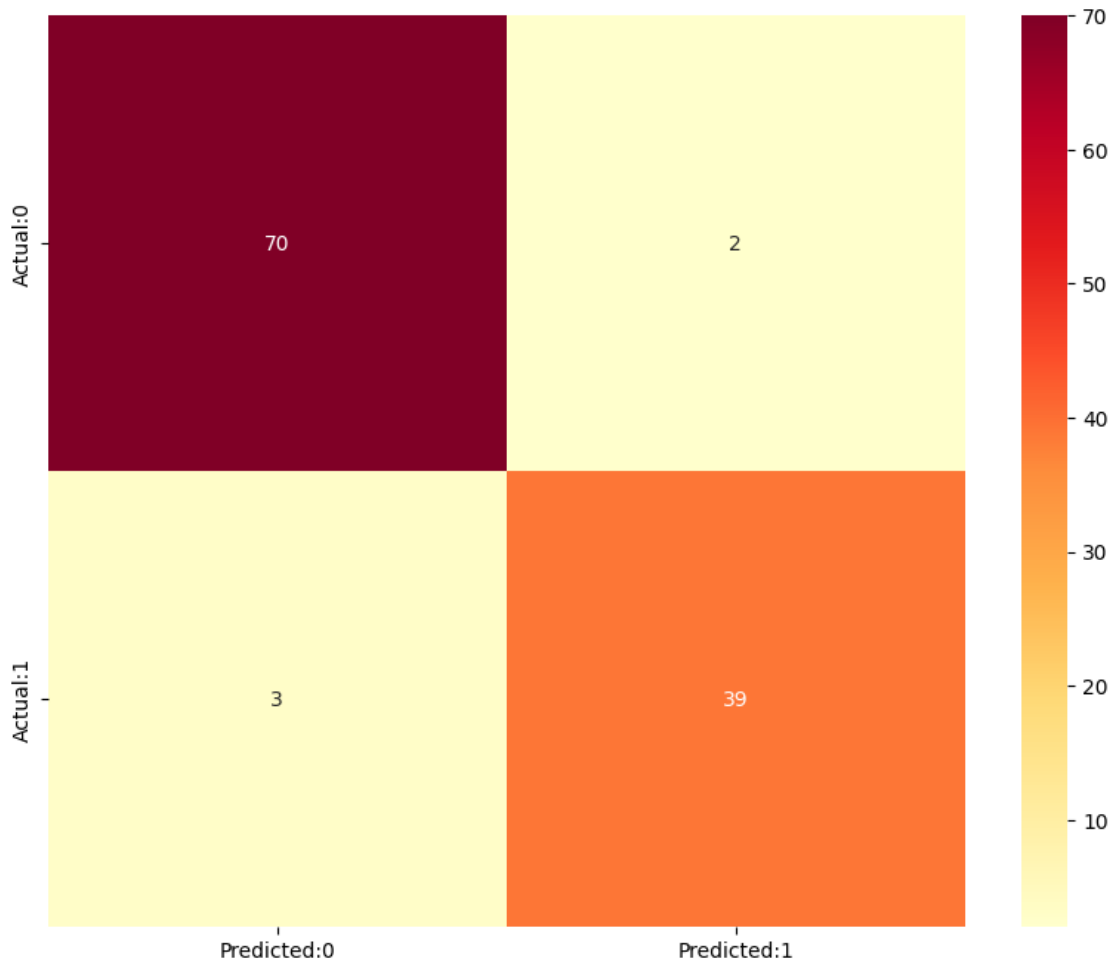
# Compare the model's predictions (y_pred) against the true answers (y_test).
# The result is a 2x2 grid counting the four possible outcomes:
#   [[true negatives,  false positives],
#    [false negatives, true positives]]
```

```
cmatrix = confusion_matrix(y_test, y_pred)

# Wrap the raw numbers in a labelled DataFrame so the heatmap
# shows meaningful row/column names instead of just 0 and 1.
# Rows = what the tumour actually was; columns = what the model said.
conf_matrix = pd.DataFrame(data=cmatrix,
                           columns=['Predicted:0', 'Predicted:1'],
                           index=['Actual:0', 'Actual:1'])

# Draw the confusion matrix as a heatmap.
# annot=True prints the count inside each cell; fmt='d' formats them as whole␣
 ↪numbers
plt.figure(figsize=(10, 8))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='YlOrRd')
plt.show()

# Print the raw matrix as text, then the overall accuracy -
# the proportion of all 114 test tumours the model classified correctly.
# Remember from the course: accuracy alone is not enough for medical data,
# which is why the next step looks at precision and recall.
print(f"The confusion matrix is:\n{cmatrix}")
print('Accuracy:', accuracy_score(y_test, y_pred))
```




```
The confusion matrix is:
[[70  2]
 [ 3 39]]
Accuracy: 0.956140350877193
```

0.0.3 Confusion Matrix explained

We evaluate the model on the test set. In this confusion matrix, **rows represent actual classes** and **columns represent predicted classes**, in the same class order:

$$\begin{pmatrix} 70 & 2 \\ 3 & 39 \end{pmatrix}$$

The diagonal contains the correct predictions: $\mathbf{70 + 39 = 109}$. The off-diagonal entries contain the errors: $\mathbf{2 + 3 = 5}$.

0.0.4 Accuracy explained

Accuracy is the proportion of predictions that are correct:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{70 + 39}{70 + 2 + 3 + 39} = \frac{109}{114} \approx 0.95614$$

The model correctly classified **109 of the 114 test cases**, giving an accuracy of approximately **95.61%**.

Accuracy counts both types of error equally. The confusion matrix also shows which actual classes were confused; interpreting these errors as false positives or false negatives requires knowing which class is designated positive.

Beyond accuracy

In a medical problem, accuracy is not enough — the two kinds of error have very different costs. Precision and recall, show how the model performs on the malignant class specifically.

0.0.5 Precision, recall, F1-score and support

These metrics describe performance **for each class separately**. To explain them for the malignant class, treat **malignant as positive** and **benign as negative**, regardless of their numerical labels.

- **True positive (TP):** a malignant case correctly predicted as malignant.
- **False positive (FP):** a benign case incorrectly predicted as malignant.
- **False negative (FN):** a malignant case incorrectly predicted as benign.
- **True negative (TN):** a benign case correctly predicted as benign.

Precision: how reliable are malignant predictions? Of all cases **predicted as malignant**, precision measures the proportion that are actually malignant:

$$\text{Precision} = \frac{TP}{TP + FP}$$

High precision means fewer false alarms among malignant predictions.

Recall: how many malignant cases are detected? Of all cases that **are actually malignant**, recall measures the proportion correctly identified:

$$\text{Recall} = \frac{TP}{TP + FN}$$

High recall means fewer malignant cases are missed. Recall for the positive class is also called **sensitivity**.

F1-score: combining precision and recall The F1-score is the **harmonic mean** of precision and recall:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

A high F1-score requires both precision and recall to be high. However, F1 does not encode the different real-world costs of false positives and false negatives, and it does not include true negatives.

Support: how many actual cases belong to the class? Support is the number of examples whose **true label** belongs to the class:

$$\text{Support}_{\text{malignant}} = TP + FN$$

$$\text{Support}_{\text{benign}} = TN + FP$$

Support is a count, not a performance score. It helps us interpret how many examples each class's metrics are based on.

Reading the classification report Each row treats its named class as the class of interest. Therefore, the malignant row measures detection of malignant cases, while the benign row measures detection of benign cases.

- **Macro average:** the unweighted mean across classes, giving each class equal importance.
- **Weighted average:** the mean weighted by each class's support, giving larger classes more influence.

Precision, recall and F1 range from **0 to 1**, with higher values indicating better performance. If a denominator is zero, the metric is undefined mathematically; scikit-learn handles this according to its `zero_division` setting.

[23]:
```
# per-class marking: precision (of those flagged, how many were right),
# recall (of the real cases, how many were caught), and their balance (f1)
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred, target_names=['Benign (0)',␣
 ↪'Malignant (1)']))
```

```
              precision    recall  f1-score   support

   Benign (0)       0.96      0.97      0.97        72
Malignant (1)       0.95      0.93      0.94        42

     accuracy                           0.96       114
    macro avg       0.96      0.95      0.95       114
 weighted avg       0.96      0.96      0.96       114
```

0.0.6 Interpreting the classification report

The test set contains **114 cases: 72 benign and 42 malignant**. The report rounds scores to two decimal places.

Benign (class 0)

- **Precision = 0.96:** Of the 73 cases predicted as benign, 70 were actually benign: $70/73 \approx 95.89\%$. The remaining **3 were malignant cases incorrectly predicted as benign**.
- **Recall = 0.97:** The model correctly identified 70 of the 72 actual benign cases: $70/72 \approx 97.22\%$. The other **2 were incorrectly predicted as malignant**.
- **F1-score = 0.97:** The harmonic mean of benign precision and recall is approximately 0.966.
- **Support = 72:** There are 72 actual benign cases in the test set.

Malignant (class 1)

- **Precision = 0.95:** Of the 41 cases predicted as malignant, 39 were actually malignant: $39/41 \approx 95.12\%$. The other **2 were false positives**.
- **Recall = 0.93:** The model detected 39 of the 42 actual malignant cases: $39/42 \approx 92.86\%$. It **missed 3 malignant cases**, which are false negatives.
- **F1-score = 0.94:** The harmonic mean of malignant precision and recall is approximately 0.940.
- **Support = 42:** There are 42 actual malignant cases in the test set.

Overall accuracy The model correctly classified **109 of 114 cases**:

$$\text{Accuracy} = \frac{70 + 39}{114} \approx 95.61\%$$

The report displays this as **0.96** after rounding.

Macro and weighted averages **Macro averages** give the two classes equal weight, regardless of their sizes. For example, macro recall is:

$$\text{Macro recall} = \frac{70/72 + 39/42}{2} \approx 95.04\%$$

This is also the **balanced accuracy** for this binary classification task.

Weighted averages weight each class's score by its support: 72 benign cases and 42 malignant cases. Consequently, benign performance has more influence on these averages.

The support of **114** on the summary rows is the total number of test cases, not an additional group.

What should we notice? The model performs well overall on this test set, but **95.61% accuracy does not mean it detects 95.61% of malignant cases**. Its malignant recall is lower, at **92.86%**.

The most important distinction is between the **2 benign cases flagged as malignant** and the **3 malignant cases predicted as benign**. Accuracy combines these five errors into one number, while the class-specific metrics reveal their different directions.

These results describe this particular test set; they do not by themselves establish suitability for clinical use.

Step 13 Validate the data

[24]:
```
# instead of a hard 0/1 answer, ask the model HOW SURE it is:
# two probabilities per tumour (benign, malignant) that add up to 1
y_pred_prob = classifier.predict_proba(X_test)
y_pred_prob_df = pd.DataFrame(data=y_pred_prob, columns=['Probability of no␣
 ↪Malignancy(0)','Probability of Malignancy (1)'])
#y_pred_prob_df.head() # Simple display of the first rows

# Fancy display of the rows with more information. It also uses the same rows␣
 ↪we selected before with 'rows_to_show'.
threshold = 0.50
# rows_to_show = slice(16, 23)

classes = list(classifier.classes_)
benign_prob = y_pred_prob[rows_to_show, classes.index(0)]
malignant_prob = y_pred_prob[rows_to_show, classes.index(1)]

# Select actual labels by position, matching the probability rows.
actual = np.asarray(y_test)[rows_to_show]
predicted = (malignant_prob > threshold).astype(int)

case_numbers = np.arange(1, len(y_pred_prob) + 1)

preview = pd.DataFrame({
    'Test case': case_numbers[rows_to_show],
    'Benign probability': benign_prob,
    'Malignant probability': malignant_prob,
    'Actual class': np.where(actual == 1, 'Malignant', 'Benign'),
    f'Predicted class at {threshold:.2f}': np.where(
        predicted == 1, 'Malignant', 'Benign'
    ),
    'Correct?': np.where(predicted == actual, 'Yes', 'No'),
})

probability_columns = ['Benign probability', 'Malignant probability']

display_table(
    preview,
    formats={column: '{:.4%}' for column in probability_columns},
    latex_formats={
        column: lambda value: f'{value:.4%}'.replace('%', r'\%')
        for column in probability_columns
    },
    html_style=correctness_html,
```

```
    latex_style=correctness_latex,
    style_columns=['Correct?'],
    fit_to_page=True,
    na_rep='-',
)
```

Test case	Benign probability	Malignant probability	Actual class	Predicted class at 0.50	Correct?
17	99.9931%	0.0069%	Benign	Benign	Yes
18	93.2480%	6.7520%	Benign	Benign	Yes
19	99.8788%	0.1212%	Benign	Benign	Yes
20	48.1029%	51.8971%	Benign	Malignant	No
21	0.2003%	99.7997%	Malignant	Malignant	Yes
22	0.0566%	99.9434%	Malignant	Malignant	Yes
23	0.0000%	100.0000%	Malignant	Malignant	Yes

What did we find?

The model scored 96% accuracy — but the confusion matrix tells the fuller story: of 42 malignant tumours in the test set, the model caught 39 and missed 3 (false negatives), while raising 2 false alarms. That's a recall of 0.93 for the malignant class. As the course's skin cancer example teaches, in medical screening the false negatives are the costly errors — which is why we judge this model on precision and recall, not accuracy alone. It also shows why results from a single train/test split should be treated with care: a different random split of this same data gives slightly different numbers, which is exactly the problem cross-validation is designed to smooth out.

0.1 Extra learning

What if we changed the threshold to something that produces no false negatives?

[25]:
```
# This contains the benign class and the malignant class
classes = list(classifier.classes_)
# We get the malignant class probabilities
malignant_prob = y_pred_prob[:, classes.index(1)]
# The probabilities converted to 0 and 1 as they stand with the default␣
 ↪threshold
actual = np.asarray(y_test)

'''Find the lowest predicted probability among actual malignant cases.
There are 3 steps:
1. actual == 1 identifies the actual malignant cases
2. malignant_prob[actual == 1] selects the predicted probabilities for the␣
 ↪actual malignant cases
3. .min() finds the lowest predicted probability among actual malignant cases
'''
lowest_malignant_prob = malignant_prob[actual == 1].min()

# Move just below it so that the strict > comparison includes that case.
# np.nextafter returns the next representable floating-point value after the␣
 ↪first argument in the direction of the second argument.
threshold = np.nextafter(lowest_malignant_prob, -np.inf)
```

```
predicted = (malignant_prob > threshold).astype(int)

false_negatives = np.sum((actual == 1) & (predicted == 0))
false_positives = np.sum((actual == 0) & (predicted == 1))

print(f"Threshold: {threshold:.17g}")
print(f"False negatives: {false_negatives}")
print(f"False positives: {false_positives}")
```

```
Threshold: 0.10979991648985102
False negatives: 0
False positives: 11
```

Recalculate the predictions using the new `threshold`, then pass them to `classification_report`:

[26]:
```
# Use the new threshold instead of classifier.predict(), which uses its default.
malignant_column = list(classifier.classes_).index(1)
# The line below gets the column with the probalities for the malignant class␣
 ↪(1)
malignant_prob = y_pred_prob[:, malignant_column]
new_predictions = (malignant_prob > threshold).astype(int)

print(f"Threshold: {threshold:.17g}")
print(f"Accuracy: {accuracy_score(y_test, new_predictions):.4f}")

print(classification_report(
    y_test,
    new_predictions,
    labels=[0, 1],
    target_names=['Benign (0)', 'Malignant (1)'],
    digits=4,
    zero_division=0,
))

print("Confusion matrix:")
print(confusion_matrix(y_test, new_predictions, labels=[0, 1]))
```

```
Threshold: 0.10979991648985102
Accuracy: 0.9035
              precision    recall  f1-score   support

   Benign (0)     1.0000    0.8472    0.9173        72
Malignant (1)     0.7925    1.0000    0.8842        42

     accuracy                         0.9035       114
    macro avg     0.8962    0.9236    0.9008       114
 weighted avg     0.9235    0.9035    0.9051       114

Confusion matrix:
```

```
[[61 11]
 [ 0 42]]
```

0.1.1 Effect of lowering the classification threshold

Lowering the malignant probability threshold from **0.50 to approximately 0.1098** detected every malignant case in this test set, but increased false positives.

Metric	Threshold 0.50	Threshold 0.1098
Malignant cases detected	39/42	42/42
False negatives	3	0
False positives	2	11
Malignant recall	92.86%	100%
Malignant precision	95.12%	79.25%
Malignant F1-score	0.9398	0.8842
Overall accuracy	95.61%	90.35%

The new confusion matrix is:

$$\begin{pmatrix} 61 & 11 \\ 0 & 42 \end{pmatrix}$$

Rows represent actual classes and columns represent predicted classes, ordered as **benign (0), malignant (1)**:

- **61 benign cases** were correctly classified.
- **11 benign cases** were incorrectly flagged as malignant.
- **No malignant cases** were missed.
- **42 malignant cases** were correctly identified.

The lower threshold detected **3 additional malignant cases**, at the cost of **9 additional false positives**. Recall improved, while precision, F1-score and overall accuracy decreased.

Evaluation limitation: This threshold was selected using these test labels. Its 100% malignant recall therefore describes threshold exploration on this dataset, not an independent evaluation or a guarantee about future cases. For a final evaluation, choose the threshold using validation data and then assess it on an untouched test set.